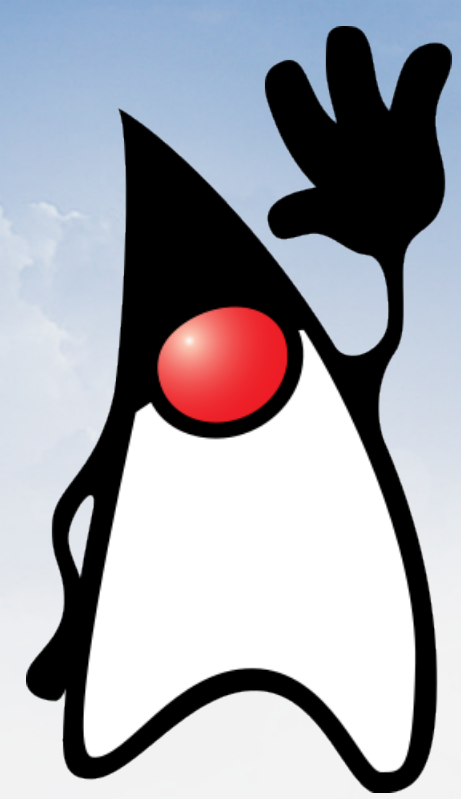




File Operations

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

Read/ Write files

- We will see examples to read/write the file line by line
- For all examples we will need to
 - `import java.io.*;`
- For modern syntaxes we will also need to
 - `import java.nio.file.*;`
- Java has `BufferedReader/Writer` for that.
 - Although it is a good practice, closing the `BufferedReader/Writer` is not mandatory. GC handles them all.
- `StringTokenizer` is very useful
- `Scanner` class is only for reading in basic level
 - I do not suggest that!
 - `Scanner` class is for getting keyboard input.



Read Line by Line classic approach

```
public static void readFile(String fname){  
    BufferedReader br=null;  
    try{  
        br=new BufferedReader(new FileReader(fname));  
        String newline=null;  
        while ((newline = br.readLine()) != null) {  
            System.out.println(newline);  
        }//end of while  
        br.close(); //not mandatory  
    }  
    catch(Exception e){  
        e.printStackTrace();  
    }//end of try catch  
}//end of method
```

Before the method definition in the class
import java.io.*; should be added.

Read all file Line by Line, Java 8

```
import java.nio.file.Files;
import java.nio.file.Paths;
import java.io.*; //for IOException
import java.util.List;

public class rfmodern{
    public static void ReadFile(String fname){
        try{ // we will be reading the whole file to a list
            List<String> allLines = Files.readAllLines(Paths.get(fname));
            for (String line : allLines) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        } //end of try catch
    } //end of method
} //end of class
```

Read file Line by Line, Java Stream syntax

```
import java.nio.file.Files;
import java.nio.file.Paths;
import java.io.*; //for IOException
import java.util.List;
public class rfmodern{
    public static void ReadFile(String fname){
        Path pathToRead = Paths.get(fname);
        if(!Files.exists(pathToRead)) { // Check if file exists
            System.out.println("file doesn't exist"); //this check is optional
        }else {
            try{
                Files.lines(pathToRead).forEach(line -> {
                    // Process the String line here!!
                    System.out.println(line);
                });
            }catch(Exception e) {
                e.printStackTrace();
            }
        }
    } //end of if
} //end of method
} //end of class
```



Write Line by Line

```
public static void WriteFile(String fname){  
    BufferedWriter bw; // why is this line outside of try catch?  
    try{  
        bw=new BufferedWriter(new FileWriter(fname));  
        bw.write("Some text here"+System.lineSeparator());  
        bw.write("another text here"+"\\n");  
        bw.close();  
    }catch(Exception e){  
        e.printStackTrace();  
    }//end of try catch  
  
    }//end of method
```

If the file exists, this will overwrite the file.

System.lineSeparator() is “\\n”



append to File

```
public static void WriteFile(String fname){  
    BufferedWriter bw; // why is this line outside of try catch?  
    try{  
        bw=new BufferedWriter(new FileWriter(fname,true));  
        bw.write("Some text here"+System.lineSeparator());  
        bw.close();  
    }catch(Exception e){  
        e.printStackTrace();  
    }//end of try catch  
  
}//end of method
```

If the file exists, this will not overwrite but append to the end of the file.

StringTokenizer

```
public static void main(String[] args){
    String lineString = "this is a string to get tokenized";
    StringTokenizer st = new StringTokenizer(lineString);
    //default constructor assumes
    // the delimiter is empty space (space,newline etc..)

    while( st.hasMoreTokens() ) {
        String token = st.next();
        // whenever you call next() it gives next token
        // Do NOT call st.next() again here.

    } //end of while
}
```

this
is
a
string
to
get
tokenized

import java.util.StringTokenizer; is needed to use String Tokenizer. It is quite useful to parse a String. String tokenizer has a cursor for showing the next available token. The cursor moves forward whenever next is called. It never moves back. Please try calling st.next() again inside.

String split

```
public static void main(String[] args){  
    String lineString = "this is a string to get tokenized";  
    String[] arrOfStr = lineString.split(" ");  
    String token=null;  
    for (int i=0; i < arrOfStr.length; i++){  
        token=arrOfStr[i];  
        System.out.println(token);  
    }//end of for  
}
```

this
is
a
string
to
get
tokenized

No import is needed to use split. However split method expects a regular expression. Therefore when you want to split using special characters (./*/?/+/-) you will be having problems. You will need to escape character. For example, to split a string on a period (.), use str.split("\\.").

```
package freadEx;
import java.io.*; //necessary for file operations
import java.util.*;
```

```
public class Fread{
    public static ArrayList<String[]> readFile(String fname){
        BufferedReader br=null;
        ArrayList<String[]> options=new ArrayList<>();
        try{
            br=new BufferedReader(new FileReader(fname));
            String newline=null;
            while ((newline = br.readLine()) != null) {
                if(!newline.substring(0,2).equals("//")){
                    options.add(newline.split(":"));
                }//end of if
            }//end of while
            br.close();
        }catch(Exception e){
            e.printStackTrace();
        }
        return options;
    }
}
```

example.txt

```
//comment here
username:admin
password:123
//another comment here
```

commented lines will be ignored

split method will return a String array

```
public static void main(String[] args) {
    ArrayList<String[]> arr=readFile("example.txt");
    for(String[] el:arr) {
        System.out.println(el[0]+" "+el[1]);
    }//end of for
} //end of main method
```

read file with Scanner

```
public ArrayList<String> getFileContents(String fname){  
    ArrayList<String> arr = new ArrayList<>();  
    File file = new File(fname);  
    Scanner scan=null;  
    try {  
        scan = new Scanner(file);  
        while (scan.hasNext()) {  
            String value= scan.next();  
            System.out.println(value);  
            arr.add(value);  
        }//end of while  
    }  
    catch (FileNotFoundException e) {  
        e.printStackTrace();  
    }  
    scan.close();  
    return arr;  
}
```

This code will read the file and tokenize the file with empty spaces.

test 101 here
some other line

test
101
here
some
other
line

read file with Scanner line by line

```
public ArrayList<String> getFileContents(String fname){
    ArrayList<String> arr = new ArrayList<>();
    File file = new File(fname);
    Scanner scan=null;
    try {
        scan = new Scanner(file);
        while (scan.hasNextLine()) {
            String line= scan.nextLine();
            System.out.println(line);
            arr.add(value);
        }//end of while

    }
    catch (FileNotFoundException e) {
        e.printStackTrace();
    }
    sc.close();
    return arr;
}
```

This code will read the file and tokenize the file with new lines.

test 101 here
some other line

test 101 here
some other line

How can we write file with Scanner class?

We cannot write file with Scanner

File Read/Write

- Closing the BufferedReader/Writer is **not mandatory**. Java detects an open stream and closes automatically.
- Closing the Scanner is also **NOT mandatory**. Java will close it automatically.
- It is a good practice to close manually.
- In other words, if we do not close the bufferedReader/Writer or scanner, we will not get a compilation error.

Wrapper Classes

- Integer, Double, Boolean, Float
 - The ones whose first letter is in Capital
- The constructors are not callable.
 - It is deprecated as Integer class caches the numbers for fast access
 - ~~Double d=new Double(3.5);~~
- Wrapper classes are immutable, also final.
- Integer num= Integer.valueOf(5); //For creating an Integer object
- int num=Integer.parseInt("5"); // For parsing integer
- Why do we have a class which has only one number inside?
 - Why is it created?

Wrapper Classes

Wrapper Class	Method to Retrieve Primitive Values
Integer	intValue()
Float	floatValue()
Double	doubleValue()
Character	charValue()
Boolean	booleanValue()

```
Float ftemp = Float.valueOf(3.4);  
float val = ftemp.floatValue();
```

```
Double dtemp = Double.valueOf(3.4);  
double val = dtemp.doubleValue();
```

Autoboxing and Unboxing

- Autoboxing - Automatic conversion of primitive data type into its corresponding wrapper class

arr.add(3); converted to *arr.add(Integer.valueOf(3));*

- Unboxing - Automatic conversion of wrapper class into its corresponding primitive data type

Integer temp=Integer.valueOf(3);

int num=temp; // temp.intValue(); is written by compiler

- Autoboxing and unboxing allows programmers to write cleaner code
- This was being done by compiler when we were using ArrayList

Revisiting ArrayList and confession

- Do you remember remove methods of ArrayList?
- For integer element removal
 - We need to call it as *myArrayList.remove(Integer.valueOf(4));*
 - *We were giving an Integer object*
- For removing an element at position 4
 - We need to call it as *myArrayList.remove(4);*
 - We were giving an int for the position
- Then why didn't we give Integer object while adding
 - *myArrayList.add(4);* was correct and working!

```
package wrapperp;  
import java.util.ArrayList;  
public class ArrayListTester {
```

```
    public static void main(String[] args) {  
        Integer num=Integer.valueOf(4);  
        int val=55;  
        ArrayList<Integer> arr=new ArrayList<>();  
  
        arr.add(num); //adding an Integer object  
        arr.add(Integer.valueOf(6));  
        arr.add(99);  
        // above is converted to arr.add(Integer.valueOf(99));  
        arr.add(val);  
        // above is converted to arr.add(Integer.valueOf(val));  
  
        System.out.println(arr);  
  
        arr.remove(1);  
        System.out.println("after removal of pos " +1);  
        System.out.println(arr);  
  
        arr.remove(Integer.valueOf(55));  
        System.out.println("after removal of element " +55);  
        System.out.println(arr);  
    } //end of main method  
} //end of class
```

ArrayList

```
[4, 6, 99, 55]  
after removal of pos 1  
[4, 99, 55]  
after removal of element 55  
[4, 99]
```

remove(int pos)
removing the element at
position pos

remove(Object el)
removing the Object el

remove

```
public E remove(int index)
```

Removes the element at the specified position in this list. Shifts any subsequent elements to the left (subtracts one from their indices).

Specified by:

`remove` in interface `List<E>`

Overrides:

`remove` in class `AbstractList<E>`

Parameters:

`index` - the index of the element to be removed

Returns:

the element that was removed from the list

Throws:

`IndexOutOfBoundsException` - if the index is out of range (`index < 0 || index >= size()`)

remove

```
public boolean remove(Object o)
```

Removes the first occurrence of the specified element from this list, if it is present. If the list does not contain the element, it is unchanged. More formally, removes the element with the lowest index *i* such that (*o*==null ? *get(i)*==null : *o.equals(get(i))*) (if such an element exists). Returns *true* if this list contained the specified element (or equivalently, if this list changed as a result of the call).

Specified by:

```
remove in interface Collection<E>
```

Specified by:

```
remove in interface List<E>
```

Overrides:

```
remove in class AbstractCollection<E>
```

Parameters:

o - element to be removed from this list, if present

Returns:

true if this list contained the specified element

```
package wrapperp;
import java.util.ArrayList; // import java.util.*; imports all
public class ArrayListTester {
    public static void main(String[] args) {
        String rem="Java";
        ArrayList<String> arr=new ArrayList<>();

        arr.add("Now");
        arr.add("Test");
        arr.add(new String("Java"));

        //printing the ArrayList
        System.out.println(arr);

        arr.remove(1);
        System.out.println("after removal of pos " +1);
        System.out.println(arr);

        arr.remove(rem);
        System.out.println("after removal of element " + rem);
        System.out.println(arr);
    }
}
```

ArrayList

[Now, Test, Java]
after removal of pos 1
[Now, Java]
after removal of element Java
[Now]

This is ArrayList of String objects. We are giving *String* object to *remove* method to remove an object. We give *int* to *remove* method to remove a position.